

Documentación técnica del código

pipeline_modelado_secop.py

Proyecto: SECOP II - ODS 16.6
Modelado predictivo de riesgo de baja transparencia

Campo	Descripción
Archivo fuente	pipeline_modelado_secop.py
Extensión aproximada	480 líneas de código
Base de datos asociada	datos_feature_engineering.parquet
Tamaño del parquet recibido	91.1 MB
Orquestador	Prefect: @flow y @task
Librerías principales	pandas, numpy, scikit-learn, matplotlib, seaborn
Modelos evaluados	Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM y CatBoost si están instalados
Fecha de generación	1 de mayo de 2026

Este documento describe la arquitectura, la lógica de procesamiento, los artefactos generados y las decisiones de ingeniería del script de modelado. **El objetivo es dejar una referencia legible para defensa oral, mantenimiento, auditoría académica y evolución del pipeline de Machine Learning.**

Contenido

1. Propósito del script
2. Contexto del proyecto y relación con ODS 16.6
3. Insumos del pipeline
4. Visión general de la arquitectura
5. Configuración general y parámetros
6. Componentes funcionales del código
7. Flujo detallado de ejecución
8. Preprocesamiento y control de fuga de información
9. Estrategia de particionado
10. Modelos implementados
11. Métricas de evaluación y visualizaciones
12. Artefactos generados
13. Robustez, trazabilidad y buenas prácticas
14. Observaciones técnicas y riesgos
15. Ejecución recomendada
16. Conclusión técnica

1. Propósito del script

El archivo `pipeline_modelado_secop.py` implementa un pipeline completo de modelado predictivo para estimar la variable objetivo `riesgo_baja_transparencia` en procesos de contratación pública de SECOP II. A diferencia de un notebook exploratorio, el script está organizado como una pieza reproducible de ingeniería de datos y Machine Learning: prepara directorios, carga un parquet enriquecido, elimina variables con fuga de información, preprocesa variables, divide el conjunto de datos, entrena varios clasificadores, calcula métricas, genera visualizaciones y redacta un reporte automático del mejor modelo.

El problema analítico se formula como clasificación supervisada binaria: identificar procesos con señales de baja transparencia a partir de variables disponibles antes de conocer el desenlace final del proceso contractual. Esto permite orientar el modelo hacia alertas tempranas y no hacia la simple reproducción de resultados posteriores a la adjudicación.

2. Contexto del proyecto y relación con ODS 16.6

El proyecto se enmarca en el análisis de contratación pública con enfoque de Big Data y Machine Learning. La base SECOP II permite observar procesos de compra pública, sus entidades, modalidades, tiempos, valores y trazabilidad digital. Este tipo de información es pertinente para el ODS 16, especialmente la Meta 16.6, porque ayuda a estudiar instituciones eficaces, transparentes y responsables.

En la lógica del proyecto, `riesgo_baja_transparencia` no es una variable administrativa original, sino una etiqueta derivada mediante feature engineering y reglas de calidad institucional. El script documentado toma esa etiqueta como target y construye un modelo que aprende patrones predictivos asociados con mayor o menor riesgo.

3. Insumos del pipeline

Insumo	Archivo	Uso dentro del informe
Código fuente	<code>pipeline_modelado_secop.py</code>	Script orquestado con Prefect que ejecuta la etapa de modelado.
Base analítica	<code>datos_feature_engineering.parquet</code>	Archivo Parquet producido después de limpieza y feature engineering. El formato Parquet es adecuado para análisis tabular por su almacenamiento columnar, compresión y preservación de tipos.
Ejemplo de documentación	<code>documentacion_tecnica_analisis_inferencial_secop.pdf</code>	Documento de referencia usado como guía de estilo, estructura y nivel de detalle.
Material del curso	Sesiones de Big Data Tools	Soporta la justificación metodológica: CRISP-DM, pipelines, orquestación, formatos analíticos, visualización y Machine Learning.

El pipeline asume que el parquet de entrada se encuentra en la ruta `Data/Processed/Limpieza/datos_feature_engineering.parquet` dentro de la estructura del proyecto. Esta

convención conecta la etapa de feature engineering con la etapa de modelado y mantiene una arquitectura por capas.

4. Visión general de la arquitectura

La arquitectura sigue una secuencia batch: carga de datos, control de leakage, transformación, particionado, entrenamiento, evaluación y persistencia de resultados. La orquestación se implementa mediante un flujo Prefect principal y tareas decoradas con @task. Además, se añaden decoradores personalizados para medición de tiempo y validación básica de entradas.

Etapa	Función principal	Resultado
Bootstrap local	check_and_create_directories	Crea carpetas de datos procesados, reportes, gráficos, logs y modelos.
Ingesta	load_data	Lee el parquet y valida la existencia del target.
Control de leakage	remove_data_leakage	Elimina columnas posteriores al resultado o identificadores no útiles para generalización.
Preprocesamiento	preprocess_and_feature_engineering	Imputa numéricas y categóricas, crea banderas de nulos y codifica categorías.
Particionado	split_datasets	Genera train, validation y test con estratificación 60/20/20.
Evaluación	eval_model y plot_model_results	Calcula métricas y guarda matriz de confusión, ROC y PR.
Modelado	train_and_evaluate_models	Entrena una batería de modelos y guarda modelos/feature importance.
Cierre	wrap_up_and_conclude	Selecciona mejor modelo por F1, evalúa en test y escribe reporte final.

5. Configuración general y parámetros

Parámetro	Valor en el código	Interpretación
PROJECT_ROOT	Path(__file__).resolve().parents[2]	Raíz del proyecto desde donde se derivan las rutas de datos, reportes y modelos.
DATA_PARQUET_PATH	Data/Processed/Limpieza/datos_feature_engineering.parquet	Entrada esperada del pipeline.
PROCESSED_DATA_DIR	Data/Processed/Model	Salida de datasets train, validation y test, además de reportes técnicos de features.
REPORTS_DIR	Reports/Model	Carpeta de métricas, gráficos, reportes de clasificación e importancia de variables.
MODELS_DIR	Models/trained	Carpeta para serializar modelos

		entrenados.
TARGET_COL	riesgo_baja_transparencia	Variable objetivo binaria del modelo.
random_state	42	Semilla usada en particionado y modelos para reproducibilidad.
Métrica de selección	F1_score	Criterio principal para escoger el mejor modelo en validación.

6. Componentes funcionales del código

El script contiene 480 líneas y define 12 funciones principales. La estructura está organizada en importaciones, configuración global, decoradores, tareas del pipeline y flujo orquestador.

Función	Línea aproximada	Rol
timing_decorator	85	Función detectada en el script
validate_inputs	98	Función detectada en el script
check_and_create_directories	115	Función detectada en el script
load_data	134	Función detectada en el script
remove_data_leakage	151	Función detectada en el script
preprocess_and_feature_engineering	182	Función detectada en el script
split_datasets	229	Función detectada en el script
eval_model	258	Función detectada en el script
plot_model_results	271	Función detectada en el script
train_and_evaluate_models	319	Función detectada en el script
wrap_up_and_conclude	395	Función detectada en el script
main_pipeline	445	Función detectada en el script

6.1. Dependencias principales

- Prefect: orquesta el flujo mediante @flow y @task, registra estados y facilita trazabilidad de ejecución.
- pandas y numpy: manipulación tabular, lectura/escritura Parquet, tipificación e imputación.
- scikit-learn: particionado, imputación, codificación, modelos clásicos y métricas de clasificación.
- matplotlib y seaborn: generación de matrices de confusión y curvas de evaluación.
- XGBoost, LightGBM y CatBoost: modelos opcionales que se incorporan solo si la dependencia está instalada.

7. Flujo detallado de ejecución

7.1. check_and_create_directories()

Crea la estructura de carpetas necesaria antes de ejecutar cualquier operación de IO. Esta tarea evita errores por rutas inexistentes y estandariza la organización de resultados.

Línea aproximada en el archivo: 113. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.2. load_data()

Recibe la ruta del parquet, valida que exista, carga el DataFrame con pandas y comprueba que el target riesgo_baja_transparencia esté presente. Si el archivo no existe o falta la variable objetivo, detiene la ejecución con un error explícito.

Línea aproximada en el archivo: 132. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.3. remove_data_leakage()

Elimina columnas que revelarían información del futuro o del resultado final del proceso. Incluye campos como adjudicado, fecha_adjudicacion, valor_total_adjudicacion, estado_resumen, transparency_score y otras variables derivadas directamente del target. También descarta identificadores y patrones como respuestas_, proveedores_, total_ y flag_hubo_participacion.

Línea aproximada en el archivo: 148. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.4. preprocess_and_feature_engineering()

Separa variables numéricas y categóricas, crea indicadores de ausencia cuando hay nulos, imputa numéricas con mediana e imputa categóricas con la moda. Posteriormente codifica las categóricas con OrdinalEncoder y guarda un resumen de features.

Línea aproximada en el archivo: 179. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.5. split_datasets()

Divide el DataFrame en X e y, convierte el target a entero y realiza dos particiones estratificadas: 60% entrenamiento, 20% validación y 20% prueba. Los tres subconjuntos se guardan en formato Parquet.

Línea aproximada en el archivo: 226. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.6. eval_model()

Calcula Accuracy, Precision, Recall, F1-score, ROC-AUC y PR-AUC. La función usa zero_division=0 para evitar fallos cuando un modelo no predice alguna clase.

Línea aproximada en el archivo: 257. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.7. plot_model_results()

Genera y guarda la matriz de confusión, el reporte de clasificación, la curva ROC y la curva Precision-Recall para cada modelo. Esta capa convierte resultados numéricos en evidencias visuales para auditoría y presentación.

Línea aproximada en el archivo: 270. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.8. train_and_evaluate_models()

Instancia una batería de modelos, ajusta cada clasificador, evalúa en validación, guarda gráficos, almacena importancia de variables cuando el modelo la expone y serializa cada modelo entrenado en formato pickle.

Línea aproximada en el archivo: 317. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

7.9. wrap_up_and_conclude()

Consolida métricas en CSV y Excel, selecciona el mejor modelo por F1-score de validación, lo evalúa sobre test y escribe un reporte textual con conclusiones y notas de producción.

Línea aproximada en el archivo: 393. Decoradores asociados: Prefect @task cuando aplica; timing_decorator y validate_inputs en varias tareas.

8. Preprocesamiento y control de fuga de información

El control de data leakage es una de las decisiones más importantes del pipeline. En problemas de transparencia institucional, usar variables disponibles después de la adjudicación puede producir métricas artificialmente altas y modelos poco útiles para alertas tempranas. Por esa razón, el script elimina variables de resultado, metadata identificadora y patrones asociados con participación o respuestas posteriores.

Grupo excluido	Ejemplos	Justificación
Columnas de resultado	adjudicado, fecha_adjudicacion, valor_total_adjudicacion, estado_resumen	Revelan o están muy cerca del desenlace final.
Columnas derivadas del target	transparency_score, nivel_riesgo_transparencia, confianza_label	Están directamente conectadas con la etiqueta riesgo_baja_transparencia.
Metadata identificadora	id_del_proceso, referencia_del_proceso, urlproceso, nombres descriptivos	No aportan generalización directa o pueden inducir memorización.
Patrones dinámicos	respuestas_, proveedores_, total_, flag_hubo_participacion	Pueden representar información conocida después de avanzar el proceso.

Después de eliminar estas columnas, el pipeline descarta filas con target nulo. Esta decisión es necesaria porque un modelo supervisado no puede aprender ejemplos sin etiqueta.

9. Estrategia de particionado

El script utiliza particionado estratificado porque, después del control de leakage, las columnas temporales de publicación y recepción son retiradas del conjunto de entrenamiento. Al no conservar un timestamp confiable para hacer split temporal, la alternativa implementada mantiene la proporción de clases en train, validation y test.

Subconjunto	Porcentaje	Uso
-------------	------------	-----

Train	60%	Ajuste de parámetros internos de cada modelo.
Validation	20%	Comparación de modelos y selección por F1-score.
Test	20%	Evaluación final del mejor modelo seleccionado.

La estratificación es adecuada si la variable riesgo_baja_transparencia está desbalanceada, porque evita que una partición quede con muy pocos casos positivos o negativos.

10. Modelos implementados

Modelo	Configuración clave	Propósito
LogisticRegression	class_weight=balanced, max_iter=2000	Modelo lineal base, interpretable y útil como referencia.
DecisionTreeClassifier	class_weight=balanced, max_depth=10	Árbol interpretable con control de profundidad para reducir sobreajuste.
RandomForestClassifier	class_weight=balanced, n_estimators=100	Ensamble robusto para datos tabulares; permite importancia de variables.
GradientBoostingClassifier	n_estimators=100	Ensamble secuencial que suele capturar relaciones no lineales.
XGBoost	scale_pos_weight dinámico	Modelo opcional de boosting; maneja desbalance mediante peso de clase positiva.
LightGBM	class_weight=balanced	Modelo opcional eficiente para grandes tablas.
CatBoost	auto_class_weights=Balanced	Modelo opcional especialmente útil para variables categóricas, aunque aquí se codifican previamente.

La selección no se hace por preferencia manual sino por desempeño en validación. El criterio implementado es F1-score, lo que equilibra precisión y recall en un contexto donde tanto falsos positivos como falsos negativos pueden ser relevantes para auditoría.

11. Métricas de evaluación y visualizaciones

Métrica	Qué mide	Lectura práctica
Accuracy	Proporción de aciertos totales.	Puede ser engañosa con clases desbalanceadas.
Precision	Entre las alertas positivas, cuántas eran realmente positivas.	Útil para evitar demasiadas alertas falsas.
Recall	Entre los casos positivos reales, cuántos fueron detectados.	Clave si se busca no perder procesos riesgosos.

F1-score	Media armónica entre precision y recall.	Métrica principal de selección del pipeline.
ROC-AUC	Capacidad de separación general entre clases.	Resume rendimiento para distintos umbrales.
PR-AUC	Área bajo la curva Precision-Recall.	Más informativa cuando el positivo es minoritario.

Además de métricas tabulares, el script genera tres tipos de evidencia visual por modelo: matriz de confusión, curva ROC y curva Precision-Recall. Para modelos con `feature_importances_`, también guarda rankings de importancia de variables en CSV.

12. Artefactos generados

Artefacto	Descripción
Data/Processed/Model/leakage_report.csv	Lista de columnas eliminadas por leakage, metadata o patrones de riesgo.
Data/Processed/Model/feature_summary.csv	Inventario de variables restantes y tipos después del preprocesamiento.
Data/Processed/Model/train.parquet	Subconjunto de entrenamiento.
Data/Processed/Model/validation.parquet	Subconjunto de validación.
Data/Processed/Model/test.parquet	Subconjunto final de prueba.
Reports/Model/metrics_summary.csv	Resumen consolidado de métricas por modelo.
Reports/Model/metrics_summary.xlsx	Versión Excel del resumen de métricas.
Reports/Model/confusion_matrices/*.png	Matrices de confusión por modelo.
Reports/Model/roc_curves/*.png	Curvas ROC por modelo.
Reports/Model/pr_curves/*.png	Curvas Precision-Recall por modelo.
Reports/Model/classification_reports/*.txt	Reportes de clasificación de scikit-learn.
Reports/Model/feature_importance/*.csv	Importancia de variables para modelos que la soportan.
Reports/Model/best_model_report.txt	Conclusión automática del mejor modelo y evaluación en test.
Models/trained/all_models/*.pkl	Modelos serializados en pickle.

13. Robustez, trazabilidad y buenas prácticas

- El pipeline crea directorios antes de escribir resultados, reduciendo fallos por rutas inexistentes.
- La carga valida la existencia del archivo y del target antes de continuar.
- `validate_inputs` detiene tareas si recibe DataFrames vacíos.
- Los decoradores de tiempo registran la duración de funciones críticas.
- Los modelos opcionales se integran con `try/except`; si una librería no está instalada, el pipeline continúa con los modelos base.

- Cada modelo se evalúa dentro de un bloque try/except para no abortar toda la corrida por un único algoritmo fallido.
- Las salidas se organizan por tipo: datos procesados, reportes, curvas, matrices, importancia de variables y modelos entrenados.

14. Observaciones técnicas y riesgos

Aspecto	Riesgo observado	Mejora recomendada
Rutas relativas rígidas	PROJECT_ROOT usa parents[2]. Si el archivo se ejecuta fuera de la estructura esperada, puede buscar el parquet en una ubicación equivocada.	Permitir argumentos CLI --input-path y --output-dir o usar variables de entorno.
Codificación ordinal básica	OrdinalEncoder asigna números a categorías sin relación ordinal real. Algunos modelos podrían interpretar orden artificial.	Para modelos lineales, considerar OneHotEncoder o pipelines ColumnTransformer.
Imputadores no persistidos	El script no guarda el encoder ni imputadores, solo modelos.	Guardar un Pipeline completo de scikit-learn para inferencia reproducible.
Sin calibración de umbral	La decisión binaria usa el umbral por defecto del modelo.	Optimizar umbral según costo de falsos negativos/falsos positivos.
Sin validación cruzada	La selección depende de un único split estratificado.	Agregar StratifiedKFold o repeated CV para mayor estabilidad.
Serialización con pickle	pickle es práctico pero sensible a versiones y no debe cargarse desde fuentes no confiables.	Registrar versiones de librerías o usar joblib dentro de un entorno controlado.
Métrica principal única	F1 es razonable, pero puede ocultar prioridades institucionales.	Reportar también PR-AUC, recall de clase positiva y matriz de costos.

15. Ejecución recomendada

El script está preparado para ejecutarse como módulo principal siempre que la estructura del proyecto exista y el parquet esté ubicado en Data/Processed/Limpieza/.

```
python pipeline_modelado_secop.py
```

Estructura mínima esperada:

```
Proyecto/
  Data/
    Processed/
      Limpieza/
        datos_feature_engineering.parquet
    Model/
  Reports/
    Model/
  Models/
    trained/
src/.../pipeline_modelado_secop.py
```

15.1. Sugerencia de versión con argumentos CLI

Para mejorar portabilidad, se recomienda agregar argparse y permitir rutas externas. Esto evitaría depender de parents[2] y facilitaría reproducibilidad en otros computadores o entornos de despliegue.

```
python pipeline_modelado_secop.py \  
--input-path Data/Processed/Limpieza/datos_feature_engineering.parquet \  
--output-dir Reports/Model
```

16. Conclusión técnica

pipeline_modelado_secop.py constituye la etapa de Machine Learning del proyecto SECOP II - ODS 16.6. Su valor principal está en transformar una base con feature engineering en un flujo reproducible de clasificación supervisada, con control explícito de fuga de información, particionado estratificado, entrenamiento comparativo de modelos, generación de métricas, visualizaciones y reporte automático del mejor clasificador.

Desde el punto de vista de Big Data Tools, el código evidencia conceptos centrales del curso: uso de Parquet como formato analítico, estructuración por capas de salida, orquestación con Prefect, modularización por tareas, trazabilidad de artefactos, evaluación de modelos y comunicación de resultados. Desde el punto de vista del problema público, el pipeline es coherente con una estrategia de alerta temprana para detectar procesos con riesgo de baja transparencia y apoyar análisis institucionales alineados con la Meta 16.6.

La principal mejora recomendada para una versión de producción es empaquetar todo el preprocesamiento y el modelo dentro de un único Pipeline serializable, incluir argumentos de línea de comandos, registrar metadatos de ejecución y calibrar el umbral de decisión según los costos reales de auditoría.

Referencias de trabajo

- Archivo fuente: pipeline_modelado_secop.py.
- Base analítica: datos_feature_engineering.parquet.
- Documento de ejemplo: documentacion_tecnica_analisis_inferencial_secop.pdf.
- Material del curso Big Data Tools: fundamentos de Big Data, formatos Parquet, pipelines, CRISP-DM, Prefect, visualización y Machine Learning.
- Proyecto SECOP II - ODS 16.6: análisis de transparencia, trazabilidad y riesgo institucional en contratación pública.